Analyzing a Triangle Problem on Finite 3-Dimensional Lattices

Javier Khaaliq, Nathaniel Johnson

Abstract

Continuing previous work on a triangle problem on finite lattices, we generalize the work to lattices in three dimensions, utilizing both 3-dimensional modeling and theoretical validation via analytical programming to find limiting probabilities of selecting different types of triangles. Our focus was to lay the groundwork for future research and provide visualization and validation tools to assist the development of new theorems.

Keywords: Lattice geometry, computational geometry, visualization

This research was funded by a NASA Space Grant, #80NSSC25M7079

1. Introduction

In 1893, Lewis Carroll presented the following question, previously posed by W.S.B. Woolhouse 30 years prior, in his book *Pillow Problems*: "Three marbles are thrown at random on the floor of a rectangular room, find the chance that the triangle that unites them will be acute-angled." Dr. Robert Niichel has worked alongside a few students to develop a solution to a variant of the problem focusing on lattices in $\mathbb{R}^2$ and finding the limiting probabilities of choosing different types of triangles. Continuing this work, we aim to set the foundation for the problem for lattices in $\mathbb{R}^3$. Specifically, we pose the question focusing on a discrete scenario: Choosing three random points from a $i \times j \times k$ grid, what is the probability of forming an obtuse triangle? Our methodology consists of creating two programs, one for visualization and another for counting and classifying each potential triangle, to efficiently compute and attempt to quickly find expressions for the limiting probabilities. We denote the grid size by $G_{i\times j\times k}$. The following notation will be observed:

- $R_{i\times j\times k}$ = number of possible right triangles in $G_{i\times j\times k}$
- $A_{i\times j\times k}$ = number of possible acute triangles in $G_{i\times j\times k}$
- $OB_{i\times j\times k}$ = number of possible obtuse triangles in $G_{i\times j\times k}$
- $N_{i\times j\times k}$ = number of possible non-triangles in $G_{i\times j\times k}$
- $T_{i\times j\times k}$ = total number of possible triangles in $G_{i\times j\times k}$

2. Triangle Counting Program

The goal of this program was to classify every possible triangle in a given lattice size. These finite results could then be used to check the theoretical theorems that researchers are developing. The program uses vectors between 3 points to check the angles and side lengths of the triangle. Based on these, either the law of cosines or the cosine dot product relation are used to classify the triangle as one of the following: right, acute, obtuse, or non-triangle.

2.1. Point and Triangle Generation

The program starts by generating a list off all possible points, based on the input lattice size. The first point is classified as (1,1,1), and additional points increase by 1 in each direction until the input lattice size (l,m,n). After generating points, the program starts iterating through possible combinations and creating triangles.

2.2. Triangle Classification

2.2.1. Dot Product

As mentioned previously, two types of classification is available. Classification based on the dot product was the first developed. It uses the following equation to find θ, the angle between two vectors.

$$cos(\theta) = \frac{u \cdot v}{||u|| \times ||v||}$$

We know that $cos(\theta)$ is positive for $0 \leq \theta < \pi/2$, negative for $\pi/2 < \theta < \pi$, and 0 for $\pi/2$. We can use this information to classify each triangle type based on the sign of cosine, skipping the norm calculations of u and v. However, since $cosine(\theta)$ is 1 for π, the norm calculations are required to classify a

non-triangle. The program checks the following equality for non-triangle classification:

$$||u|| \times ||v|| = \pm\ u \cdot v$$

For a given combination of points, the dot-product program iterates through all angles of a triangle and determines the polarity of the angle from the cosine equation. If the angle is found to be 0, non-triangle, or negative, the triangle is classified accordingly. If the angle is positive (acute), the program moves to the next angle to check. In the case where all angles are found to be positive, the triangle is classified as acute.

2.2.2. Law of Cosines

Following the production of the dot product program, a very similar program was made that uses the Law of Cosines for classifications:

$$c^2 = a^2 + b^2 - 2ab \cdot cos(\theta)$$

where a, b, and c are the side lengths of the triangle and θ is the angle between a and b. In particular, the program uses this equation for its calculations:

$$\theta = \arccos\left(\frac{c^2 - a^2 - b^2}{-2ab}\right)$$

Following the generation of the point in the lattice, the cosine program iterates through all possible point combinations, similarly to the dot product program. However, as shown above, the function arccos is used to determine θ. θ is then compared directly to $\pi/2$ to determine the type of angle. This difference comes with two primary issues: norm calculation and float truncation.

In order to use the Law of Cosines, all three vector lengths need to be calculated. For larger vectors, this is a computationally heavy task, slowing down the program. For classification using the dot product, checking for a right angle occurs before the vector calculation, which means it could potentially skip up to three norm calculations.

Secondly and more importantly, the Python function arccos returns a float, which is truncated to 16 digits. The function is not accurate to this level of precision; since determining a right angle requires θ to be *exactly* $\pi/2$, checking for 16 digits of accuracy caused the program to misclassify right triangles as obtuse or acute. To obtain the expected results, θ was

rounded to 10 digits. However, vectors with larger norms may still result in inaccurate classifications. So, it is recommended to use the dot product program over the cosine program to get the most accurate results.

3. Visualization Program

The goal of this program was to provide a way for visual analysis of triangles and non-triangles of interest on a 3-dimensional lattice. This program was written using Python and utilizing the Plotly and Dash libraries.

3.1. The 3-dimensional Lattice Grid

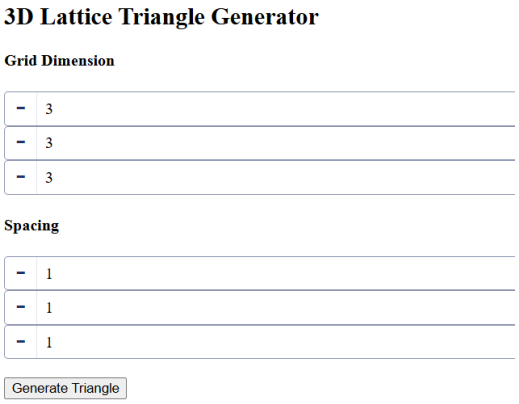

Figure 3.1: Grid Settings

The finite lattice grid is embedded in $\mathbb{R}^3$. We are generating the set

$$\{(i \cdot dx, j \cdot dy, k \cdot dz) | 0 \leq i < nx, 0 \leq j < ny, 0 \leq k < nz\}$$

where nx, ny, nz are the number of points in each direction and dx, dy, dz are the spacing between each points. The program takes the number of spacing for each direction and expands the lattice accordingly. This creates a rectangular lattice unless $dx = dy = dz$. Additionally, the total number of points in the grid is given by $nx \cdot ny \cdot nz$. As seen above in 3.1, the grid dimension takes in user-input for the values of nx, ny, nz respectively from top to bottom. In spacing, the user will enter the desired values for dx, dy, dz.

Figure 3.2: Manual Mode for User-Inputted Triangles

The automatic mode for triangle generation is Random. This selects 3 distinct lattice points uniformly at random. This sample is given as

$$\{A, B, C\} \subset G_{x \times y \times z}, A \neq B \neq C.$$

There is a Manual option that allows for the user entering the specific coordinates for a triangle to be generated. If the user accidentally chooses points that lay outside the grid, a warning is prompted so that the user can make the appropriate corrections. Altogether, using this information, the program then turns to computing the figure's geometry and classification.

Take $\{A, B, C\}$ to be a given subset of the lattice. The side lengths are computed by using the standard Euclidean distance in $\mathbb{R}^3$, that is, $AB = ||B - A||$.This gives us two vectors which we can use to compute the area of the triangle by the cross product formula:

$$Area = \frac{1}{2}||AB \times AC||.$$

Triangle classification is done by using the Law of Cosines. The program compares c^2 with $a^2 + b^2$ to determine the type of triangle generated. If $c^2 < a^2 + b^2$, all angles are acute. If $c^2 = a^2 + b^2$, the triangle is a right triangle with angle $C = \frac{\pi}{2}$. If $c^2 > a^2 + b^2$, then the triangle is obtuse with angle $C > \frac{\pi}{2}$. For non-triangles, the program detects if $AB \times AC$=0 which tells us that all three points lie on a line.

4. $2 \times 2 \times 2$ Case

The first lattice analyzed was the $2 \times 2 \times 2$ case. Immediately, one result was made evident given the size of the lattice.

Lemma 4.1. *The number of non-triangles in the* $2 \times 2 \times 2$ *case is*

$$N_{2\times2\times2} = 0.$$

Similarly, we found that there were also no obtuse triangles that could appear in a lattice of this size.

Lemma 4.2. *The number of obtuse triangles in the* $2 \times 2 \times 2$ *lattice is*

$$OB_{2\times2\times2} = 0.$$

This leaves the number of right triangles and acute triangles to be found. Right triangles could be found by taking any two collinear points and a third point elsewhere. Hence, they appeared on the "faces" of the lattice but could also be formed diagonally.

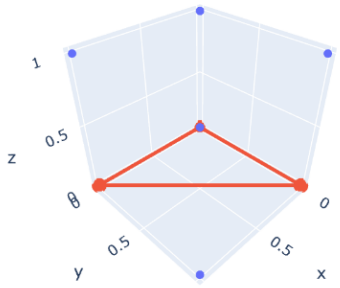

Figure 4.1: A right triangle in the $2 \times 2 \times 2$ lattice

Counting these types of triangles, we found that

Lemma 4.3. *The number of right triangles in the* $2 \times 2 \times 2$ *case is*

$$R_{2\times2\times2} = 48.$$

Carefully, we were able to find that the only type of acute triangles which were allowed in the $2\times2\times2$ case were equilateral triangles. Each triangle was formed by taking one point as the central point and choosing one point from its immediate left and another from its immediate right as pictured below.

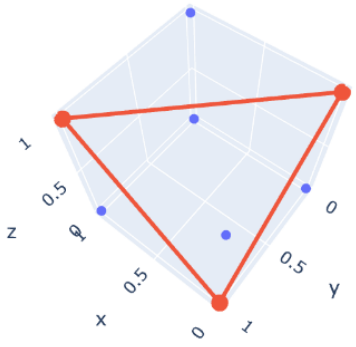

Figure 4.2: An acute triangle in the $2 \times 2 \times 2$ lattice

Forming triangles this way is quite limited. This means that the number of acute triangles in the $2 \times 2 \times 2$ case is much lower than that of the right triangles in this lattice. Specifically,

Lemma 4.4. *The number of acute triangles in the* $2 \times 2 \times 2$ *case is*

$$A_{2\times2\times2} = 8.$$

Altogether, we find that the number of triangles in the $2 \times 2 \times 2$ case is given by the following:

Theorem 4.5. *The number of triangles in the* $2 \times 2 \times 2$ *lattice is*

$$T_{2\times2\times2} = A_{2\times2\times2} + R_{2\times2\times2} = 56$$

5. $2 \times 2 \times 3$ Case

We move onto the $2 \times 2 \times 3$ case, which means that the lattice takes more of a rectangular shape compared to the $2 \times 2 \times 2$ case. One of the first things we noticed was that we could symmetrically divide the lattice into two copies of the $2 \times 2 \times 2$ lattice. This provides us with an efficient way to calculate a large amount of the right and acute triangles present in a lattice. By Lemma 4.3, we begin with 48 right triangles, doubling this gives us 96 right triangles. However, we must account for double counting, so that eliminates 4 triangles, giving us 92 right triangles already present given

the symmetry of the lattice. Approaching this lattice similarly with Lemma 4.4, we find that the lattice has at least 16 acute triangles (note: given the conditions for an acute triangle in the $2 \times 2 \times 2$ case, double counting isn't accounted for). Now, this provides us with a decent amount of triangles in the lattice, but it does not count all possible triangles, even the right and acute triangles. The next step is to focus on the triangles which require the $2 \times 2 \times 3$ lattice to be constructed. We switched to using our programs as counting and verifying each type of triangle started to become difficult due to the larger number of triangles or uncertainty of correct geometric work.

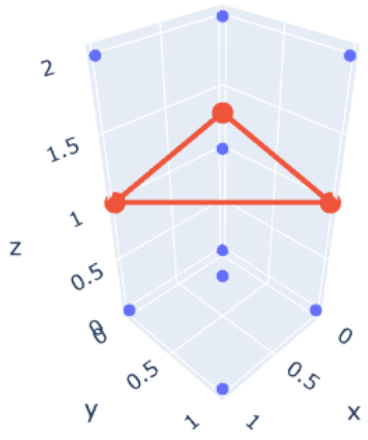

Figure 5.1: An acute triangle in the $2 \times 2 \times 3$ lattice

Lemma 5.1. *The number of acute triangles in the* $2\times2\times3$ *lattice is* $A_{2\times2\times3} = 28$.

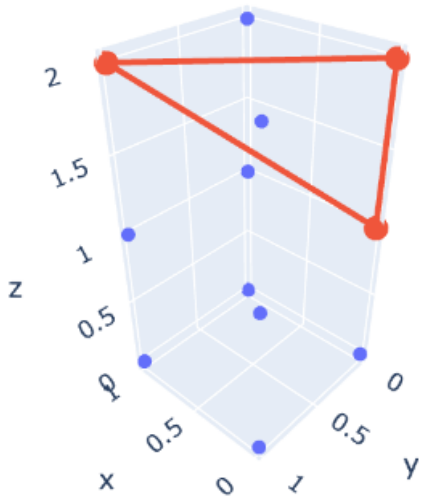

Figure 5.2: A right triangle in the $2 \times 2 \times 3$ lattice

Lemma 5.2. *The number of right triangles in the* $2\times2\times3$ *lattice is* $R_{2\times2\times3} = 156$.

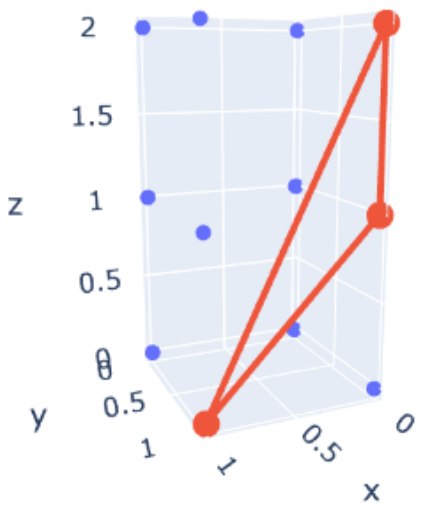

Figure 5.3: An obtuse triangle in the $2 \times 2 \times 3$ lattice

Lemma 5.3. *The number of obtuse triangles in the* $2 \times 2 \times 3$ *lattice is* $OB_{2\times2\times3} = 32$.

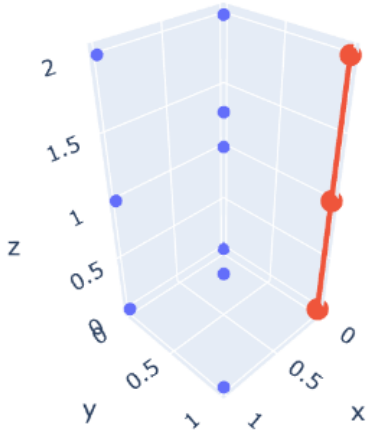

Figure 5.4: A (collinear) non-triangle in the $2 \times 2 \times 3$ lattice

Lemma 5.4. *The number of non-triangles in the* $2\times2\times3$ *lattice is* $N_{2\times2\times3} = 4$.

From Lemma 5.4, we were able to find an equation for the number of non-triangles in the $2 \times 2 \times n$ case.

Lemma 5.5. *The number of non-triangles in the* $2 \times 2 \times n$ *case is given by*

$$N_{2\times2\times2} = 4\binom{n}{3}$$

where n *is the third coordinate of the lattice.*

Following these results, progress slowed as we took time to verify these results and ensure the programs were correctly counting and classifying each type of triangle. These results were clarified more using Nathaniel's second counting program, using the Law of Cosines approach, which provided us with the same amount of triangles with the same classifications. The next step we leave for the next cohort of students is to continue in the $2\times2\times4$ case and work towards finding the limiting probabilities for each type of triangle and total number of triangles in the discrete 3-dimensional lattice. As of now, we conclude with the following:

Theorem 5.6. *The number of triangles in the* $2 \times 2 \times 3$ *lattice is*

$$T_{2\times2\times3} = 220.$$